

Kesenjangan Digital dan Kendala Infrastruktur Komputasi Faskes 3T

Fasilitas Pelayanan Kesehatan Tingkat Pertama (FKTP) seperti Puskesmas di wilayah Tertinggal, Terdepan, dan Terluar (3T) di Indonesia memegang peranan krusial sebagai garda terdepan penanganan kesehatan masyarakat [cite: 1, 2]. Namun, implementasi teknologi medis modern di wilayah-wilayah ini terhambat oleh kesenjangan digital yang ekstrem [cite: 1, 3]. Berdasarkan tinjauan sistematis terhadap tata kelola sistem informasi kesehatan, kendala utama yang dihadapi meliputi ketidakstabilan pasokan energi listrik, ketiadaan atau keterbatasan bandwidth jaringan internet, keterbatasan literasi digital tenaga kesehatan lokal, serta usangnya perangkat keras komputasi yang tersedia [cite: 1, 4, 5].

Upaya digitalisasi yang dicanangkan oleh Kementerian Kesehatan melalui integrasi platform SatuSehat menghendaki standar perangkat keras minimum tertentu [cite: 1, 6]. Namun pada kenyataannya, banyak Puskesmas terpencil yang masih menggunakan sistem komputer lama yang dialokasikan untuk mengoperasikan Sistem Informasi Manajemen Puskesmas (SIMPUS) berbasis web dengan kapasitas pemrosesan yang sangat terbatas [cite: 1, 7, 8]. Masalah geografis ini secara langsung memengaruhi ketepatan waktu penegakan diagnosis klinis, khususnya untuk penyakit paru menular seperti Tuberkulosis (TB) yang menempatkan Indonesia pada peringkat beban kasus tertinggi kedua di dunia [cite: 9, 10].

Untuk memfasilitasi kebutuhan deteksi dini tanpa ketergantungan pada konektivitas komputasi awan (*cloud computing*) yang tidak andal di daerah terpencil, model kecerdasan artifisial harus dirancang untuk dapat dieksekusi secara lokal (*on-device inference*) dengan memanfaatkan daya pemrosesan unit pemroses sentral (CPU) lokal secara efisien [cite: 4, 11, 12].

Tabel 1 menyajikan perbandingan spesifikasi komputer yang umum ditemui pada Puskesmas di Indonesia dengan persyaratan sistem aplikasi berbasis kecerdasan artifisial konvensional.

Kategori Perangkat	Prosesor (CPU)	Kapasitas RAM	Penyimpanan Medis	Hambatan Konektivitas	Sumber Acuan
Komputer SIMPUS Klasik	Intel Pentium IV / AMD Setara	128 MB - 512 MB	HDD $\geq$ 10 GB	Luring Total / Offline	[cite: 7]
Komputer SIMPUS Modern	Dual Core CPU 2.0 GHz	2 GB	HDD / SSD Menyesuaikan	Internet 1 Mbps (Sering Terputus)	[cite: 8]
Standar Minimum SatuSehat	Intel Core i3 Gen 8 /	4 GB	SSD $\geq$ 256 GB	Konektivitas Satelit / Starlink	[cite: 6, 13]

Kategori Perangkat	Prosesor (CPU)	Kapasitas RAM	Penyimpanan Medis	Hambatan Konektivitas	Sumber Acuan
	Athlon Silver				
Rekomendasi Workstation AI	Intel Xeon / Core i7 + GPU Nvidia	≥ 16 GB	SSD NVMe ≥ 1 TB	Konektivitas Awasi Konstan (Cloud)	[cite: 14, 15]

Analisis Model Kecerdasan Artifisial untuk Citra Medis dan Tantangan Penerapannya

Perkembangan teknologi pembelajaran mendalam (*deep learning*) telah menghasilkan berbagai model mutakhir untuk analisis citra medis, mulai dari klasifikasi penyakit hingga segmentasi organ [cite: 16, 17, 18]. Di sektor kesehatan global, kerangka kerja berbasis Vision-Language Models (VLM) seperti MONAI Multimodal dan model agenik VILA-M3 menawarkan kemampuan interpretasi visual radiologi 3D yang sangat komprehensif dengan mengintegrasikan citra CT, MRI, dan data rekam medis elektronik (EHR) [cite: 19, 20].

Meskipun memiliki akurasi diagnostik yang sangat tinggi, model-model besar ini menuntut alokasi memori grafis (VRAM) GPU yang sangat masif; model VILA-M3 versi 8B memerlukan sekitar 18 GB memori GPU, sedangkan versi 13B membutuhkan lebih dari 30 GB memori GPU [cite: 14]. Model segmentasi anatomi seperti VISTA3D juga memerlukan setidaknya 12 GB memori GPU [cite: 14]. Kebutuhan sumber daya ini mustahil dipenuhi oleh komputer faskes di daerah 3T yang umumnya tidak memiliki GPU diskret [cite: 1, 15].

Untuk klasifikasi radiografi dada (X-ray), pustaka terbuka seperti TorchXRyVision (XRV) menyediakan alternatif model berbasis arsitektur DenseNet-121 yang telah dilatih menggunakan jutaan gambar dari berbagai dataset radiologi dunia [cite: 17, 21, 22]. DenseNet-121 dari TorchXRyVision mampu mengidentifikasi hingga 18 jenis patologi organ dada secara simultan dengan kebutuhan memori runtime sekitar 1.5 GB pada CPU, menjadikannya kandidat yang jauh lebih realistis untuk dioptimalkan [cite: 14, 21, 22].

Selain itu, model ringan terdedikasi seperti LTS-Net (Lightweight Triple Permuter Split Attention Network) dan MobileNetV3-Large menunjukkan efisiensi tinggi pada komputasi tepi (*edge computing*) [cite: 18, 23, 24]. Di Indonesia, implementasi klinis sistem berbantuan komputer (CAD) seperti CAD4TB dari Delft Imaging menunjukkan sensitivitas sebesar 81.04% dan spesifisitas sebesar 63.80% pada indeks cutoff 60 untuk populasi lokal [cite: 25], didukung oleh penandatanganan kemitraan strategis dengan Kementerian Kesehatan pada Juni 2025 [cite: 9, 26].

Inisiatif lokal lainnya seperti TBScreen.AI yang dikembangkan oleh Universitas Gadjah Mada (UGM) bersama mitra internasional di bawah program KONEKSI menunjukkan akurasi awal

sekitar 64% pada pengujian di lokasi uji coba Balkesmas Klaten dan RSUD Mimika [cite: 27]. Pengembangan aplikasi penapisan lainnya, seperti CUHAS-ROBUST untuk deteksi TB resisten rifampisin, juga mengonfirmasi pentingnya model diagnostik berbiaya rendah dan andal untuk mengatasi keterlambatan rujukan di Puskesmas terpencil [cite: 28].

Tabel 2 membandingkan performa akurasi klasifikasi beberapa arsitektur convolutional neural network (CNN) standar pada citra radiografi dada sebelum dilakukan optimasi kompresi

Nama Model	Dataset Pre-trained	Metode Normalisasi	Rata-Rata Skor AUROC	Rata-Rata Skor AUPRC	Kebutuhan Parameter Grafis	Sumber Acuan
DenseNet121	Radiografi Dada (XRV)	Histogram Normalization (HN) + RGN	0.734	0.414	Tinggi / Kompleks	[cite: 29]
VGG16	ImageNet	Histogram Normalization (HN)	0.717	0.403	Sangat Tinggi / Lambat	[cite: 29]
Inception V3	ImageNet	Tanpa Transformasi	0.714	0.440	Sedang / Cukup Cepat	[cite: 29]
ResNet50	ImageNet	Histogram Normalization (HN) + RGN	0.712	0.344	Sedang / Aliran Baik	[cite: 29]
ViT (12 Blocks)	Inisialisasi Acak	Tanpa Transformasi	0.661	0.205	Kompleksitas Kuadratik	[cite: 29]

Metodologi Modifikasi Kompresi Arsitektur Model AI

Untuk mereduksi beban komputasi model dasar agar dapat dijalankan pada CPU dengan spesifikasi terbatas tanpa mendegradasi akurasi diagnostik secara drastis, diterapkan tiga pilar rekayasa kompresi model: Distilasi Pengetahuan (*Knowledge Distillation*), Kuantisasi Rendah Bit (*Low-Bit Quantization*), dan Modifikasi Kepala Klasifikasi Adaptif [cite: 30, 31, 32].

Distilasi Pengetahuan (Knowledge Distillation)

Metode ini mentransfer representasi spasial dan probabilitas kelas dari model guru (*teacher network*) yang berkapasitas besar dan berakurasi tinggi ke model murid (*student network*) yang jauh lebih kompak dan cepat [cite: 30, 33]. Sebagai contoh, pengetahuan dari model DenseNet-201 atau stasiun kerja ensemble TorchXRayVision dapat disalurkan ke model murid berbasis MobileNetV3 atau arsitektur supernet seperti OFA-595 yang memiliki kesamaan struktural dengan EEEA-Net-C2 [cite: 33, 34].

Model murid dilatih menggunakan fungsi kerugian gabungan (*hybrid loss function*) yang meminimalkan deviasi terhadap label biner asli (*hard labels*) sekaligus distribusi probabilitas halus (*soft targets*) yang dihasilkan oleh guru [cite: 33, 35, 36]. Distribusi probabilitas halus dari logit mentah model guru z_t dan murid z_s diformulasikan menggunakan fungsi softmax termodifikasi dengan parameter temperatur τ [cite: 35, 36]:

$$p_i = \frac{\exp(\tau z_{sj})}{\sum_j \exp(\tau z_{sj})} \exp(\tau z_{si})$$

Di mana $\tau > 1$ berfungsi memperhalus distribusi probabilitas keluaran sehingga murid mampu mempelajari hubungan antarkelas yang diekstrak oleh model guru [cite: 35, 36]. Persamaan fungsi kerugian total untuk melatih model murid didefinisikan sebagai kombinasi linear berikut [cite: 35, 36]:

$$L_{total} = \alpha LBCE(y, p_s) + (1 - \alpha) LKL(p_t, p_s)$$

Parameter $\alpha \in [0, 1]$ bertindak sebagai penyeimbang kontribusi antara pengawasan label nyata dan target distilasi halus [cite: 35]. Untuk meningkatkan transfer representasi, distilasi juga diterapkan pada tingkat fitur intermediate (*feature-based distillation*) dengan meminimalkan kesalahan kuadrat rata-rata (*mean squared error*) antara peta fitur guru dan murid, serta distilasi berbasis hubungan menggunakan matriks Gram untuk mentransfer korelasi spasial saluran konvolusi [cite: 16, 37].

Kuantisasi Pasca-Pelatihan (Post-Training Quantization / PTQ)

Kuantisasi mengurangi presisi numerik parameter model dari tipe floating-point 32-bit (FP32) menjadi tipe integer 8-bit (INT8) atau FP16 [cite: 11, 32, 38]. Konversi nilai dilakukan secara linear berdasarkan persamaan matematika berikut [cite: 39, 40, 41]:

$$val_{fp32} = Scale \times (val_{quantized} - Zero_point)$$

Di mana parameter *Scale* mewakili lebar langkah kuantisasi numerik dan *Zeropoint* memetakan representasi angka nol riil ke dalam domain terkompresi [cite: 39, 40, 41]. Pada komputer faskes tua yang umumnya tidak dilengkapi dengan instruksi pemrosesan vektor modern seperti *_Vector Neural Network Instructions* (VNNI) pada arsitektur x86, akumulator perkalian integer 8-bit rentan mengalami saturasi numerik [cite: 42].

Oleh karena itu, penyesuaian parameter kompresi dengan membatasi rentang nilai dinamis melalui opsi pembatasan jangkauan (`reduce_range=True`) wajib diaktifkan guna mencegah terjadinya degradasi akurasi diagnostik pasca-kuantisasi [cite: 42].

Modifikasi Kepala Klasifikasi dengan XGBoost

Dalam skenario klinis di mana model CNN pra-latih (seperti DenseNet-121 dari TorchXRayVision) telah dibekukan (*frozen weights*) untuk bertindak sebagai ekstraktor fitur statis, pendekatan modifikasi kepala klasifikasi adaptif terbukti sangat efisien [cite: 31, 43].

Lapisan klasifikasi akhir yang padat (*dense layer*) digantikan dengan pengklasifikasi berbasis Extreme Gradient Boosting (XGBoost) multi-head [cite: 31, 43].

Pipa pemrosesan ini mengekstrak vektor fitur berdimensi tinggi dari lapisan terakhir CNN yang dibekukan, lalu melatih model pohon keputusan XGBoost di atas representasi tersebut [cite: 31, 43]. Eksperimen membuktikan bahwa penggantian lapisan akhir dengan XGBoost tidak hanya menurunkan kompleksitas komputasi secara radikal selama fase inferensi, melainkan juga secara signifikan mengurangi bias demografis (jenis kelamin, usia, dan ras) yang sering kali melekat pada model medis mendalam [cite: 31, 43].

Tabel 3 menyajikan estimasi perbandingan efisiensi sumber daya dan performa diagnostik model sebelum dan sesudah penerapan kompresi hibrida.

Status Model	Arsitektur & Optimasi	Ukuran File Model	Latensi Inferensi CPU (1 Core)	Akurasi Diagnostik (TB / COVID-19)	Kebutuhan Memori Runtime
Sebelum Kompresi	DenseNet-121 (FP32)	≈ 30 MB	≈ 3.000 ms - 8.000 ms	Rata-Rata AUC: 88.9%	≈ 1.5 GB
Sesudah Distilasi	OFA-595 Student (FP32)	≈ 18 MB	≈ 1.500 ms - 2.500 ms	Rata-Rata AUC: 87.1%	≈ 750 MB
Sesudah Kuantisasi	OFA-595 Student (INT8)	≈ 4.5 MB	≈ 350 ms - 600 ms	Rata-Rata AUC: 86.8%	≈ 180 MB
Modifikasi Ekstra	KDLight (MobileNet-Based)	≈ 7.5 MB	≈ 420 ms	Klasifikasi Akurasi: 95.5%	≈ 250 MB

Optimalisasi Runtime Kompiler Medis pada Sistem CPU Lokal

Untuk mengeksplorasi performa perangkat keras CPU lokal secara maksimal di Puskesmas 3T, model terkompresi harus dijalankan menggunakan engine runtime khusus yang mengompilasi graf model ke dalam instruksi mesin tingkat rendah [cite: 11, 44].

Intel OpenVINO Toolkit dan NNCF

OpenVINO mentransformasikan graf model dari pustaka PyTorch atau ONNX ke dalam representasi intermediate (IR) biner yang efisien [cite: 38, 45]. Melalui pustaka *Intel Extension for PyTorch* (IPEX), graf model memanfaatkan instruksi perangkat keras tingkat lanjut seperti AVX-512 dan *Advanced Matrix Extensions* (AMX) yang terintegrasi pada CPU Intel modern [cite: 38].

Optimasi runtime dilakukan dengan mengatur jumlah aliran eksekusi paralel melalui parameter `NUM_STREAMS` agar bernilai sesuai dengan core CPU fisik yang tersedia, serta mematikan pinning CPU jika bersaing dengan aplikasi SIMPUS lain [cite: 44]. Penyesuaian ini menghasilkan peningkatan kecepatan pemrosesan hingga beberapa kali lipat pada pengujian server CPU lokal [cite: 38, 44].

ONNX Runtime (ORT) Execution Providers

ONNX Runtime mengadopsi representasi graf teroptimasi menggunakan format QDQ (Quantize-Dequantize) [cite: 40, 42]. Driver internal ORT menggabungkan operasi konvolusi terkuantisasi dengan layer aktivasi berikutnya secara langsung pada register CPU, sehingga menghindari overhead pemindahan data antar-cache memori [cite: 42, 46].

Untuk memastikan eksekusi yang konsisten pada berbagai varian sistem operasi di komputer faskes, integrasi library ONNX Runtime berbasis bahasa Python atau C++ direkomendasikan karena kompatibilitas multi-platformnya yang luas [cite: 42, 47].

Simulasi Batasan Perangkat Keras Komputer Faskes 3T Menggunakan Kontainer Docker

Untuk memvalidasi kelayakan operasional model AI portabel sebelum dideploy secara luas di daerah 3T, server pengujian dikonfigurasi untuk menyimulasikan profil batasan perangkat keras komputer Puskesmas melalui kontainerisasi Docker [cite: 48]. Docker menggunakan fitur *cgroups (control groups)* pada kernel Linux untuk membatasi akses RAM, CPU, dan laju transfer disk secara ketat [cite: 48, 49].

Parameter Pembatasan Sumber Daya Docker

Tabel 4 menjelaskan parameter baris perintah Docker run yang digunakan untuk mengemulasikan dua jenis komputer faskes 3T.

Parameter Docker	Emulasi Komputer Legacy (SIMPUS Klasik)	Emulasi Komputer Modern (SatuSehat)	Tujuan Emulasi Spesifikasi	Sumber Acuan
<code>--memory</code>	1g (1 Gigabyte)	4g (4 Gigabyte)	Membatasi konsumsi memori fisik maksimum guna mencegah kegagalan OOM Killer.	[cite: 48, 50]
<code>--memory-swap</code>	1g (Menonaktifkan swap)	4g (Menonaktifkan swap)	Memastikan model tidak menggunakan swap disk lambat yang	[cite: 48, 50]

Parameter Docker	Emulasi Komputer Legacy (SIMPUS Klasik)	Emulasi Komputer Modern (SatuSehat)	Tujuan Emulasi Spesifikasi	Sumber Acuan
			mendegradasi performa.	
<code>--cpus</code>	1.0 (Core tunggal)	2.0 (Core ganda)	Membatasi utilisasi daya pemrosesan inti CPU untuk mengukur performa multithreading.	[cite: 48, 50]
<code>--cpuset-cpus</code>	"0" (Hanya Core 0)	"0,1" (Hanya Core 0 dan 1)	Mengisolasi eksekusi proses model AI pada inti fisik tertentu di CPU server host.	[cite: 48, 49]
<code>--device-read-bps</code>	/dev/sda:10mb (10 MB/s)	/dev/sda:50mb (50 MB/s)	Membatasi kecepatan baca dari perangkat penyimpanan untuk meniru HDD lama.	[cite: 49, 51]
<code>--device-write-bps</code>	/dev/sda:10mb (10 MB/s)	/dev/sda:50mb (50 MB/s)	Membatasi kecepatan tulis untuk mensimulasikan bottleneck penyimpanan sekunder.	[cite: 49, 51]

Perintah Pembangunan dan Penjalanan Kontainer Simulasi

Skenario pengujian dijalankan pada server penampung menggunakan konfigurasi isolasi kontainer terdedikasi [cite: 14, 48].

Langkah A: Pembuatan Dockerfile Berbasis CPU Teroptimasi

Dockerfile berikut mengompilasi semua dependensi kompresi dan runtime di atas sistem Ubuntu mini [cite: 14].

```
# Menggunakan base image resmi dengan Python terinstal
FROM python:3.10-slim
```

```
# Instalasi paket sistem dasar untuk operasi citra medis
RUN apt-get update && apt-get install -y \
    build-essential \
    libgl1-mesa-glx \
    libglu1-mesa-dev \
    libglut-dev \
    && rm -rf /var/lib/apt/lists/*

# Pemasangan pustaka kompresi dan runtime AI
RUN pip install --no-cache-dir \
    numpy \
    opencv-python \
    onnx \
    onnxruntime \
    torchxrayvision \
    opencv-python-headless

# Menyiapkan direktori aplikasi
WORKDIR /app
COPY . /app

# Perintah default penjalanan aplikasi penapisan
CMD ["python", "run_inference.py"]
```

Pembangunan kontainer dilakukan menggunakan parameter optimasi paralel jaringan [cite: 14]:

```
docker build --network=host --progress=plain -t ai-faskes-3t:latest .
```

Langkah B: Eksekusi Kontainer Emulasi Komputer Legacy (SIMPUS Klasik)

Skenario ini meniru lingkungan ekstrem di mana aplikasi AI harus berbagi daya dengan SIMPUS lama pada CPU single-core dan RAM 1 GB, serta hard disk usang berkecepatan rendah [cite: 7, 49, 51].

```
docker run -it --rm \
    --name simulasi-faskes-legacy \
    --memory="1g" \
    --memory-swap="1g" \
    --cpus="1.0" \
    --cpuset-cpus="0" \
    --device-read-bps=/dev/sda:10mb \
    --device-write-bps=/dev/sda:10mb \
    ai-faskes-3t:latest bash
```

Langkah C: Eksekusi Kontainer Emulasi Komputer Modern (SatuSehat)

Skenario ini merepresentasikan komputer Puskesmas modern yang telah memenuhi standar minimal Kementerian Kesehatan untuk integrasi SatuSehat [cite: 6, 52].

```
docker run -it --rm \
  --name simulasi-faskes-satusehat \
  --memory="4g" \
  --memory-swap="4g" \
  --cpus="2.0" \
  --cpuset-cpus="0,1" \
  --device-read-bps=/dev/sda:50mb \
  --device-write-bps=/dev/sda:50mb \
  ai-faskes-3t:latest bash
```

Prosedur Validasi Pembatasan di Dalam Kontainer

Setelah masuk ke shell interaktif kontainer, verifikasi penerapan isolasi sumber daya cgroups wajib dilakukan secara langsung [cite: 49].

1. **Pengujian Kecepatan Tulis Disk (I/O Bottleneck):** Gunakan utilitas penulisan byte langsung untuk membypass cache sistem operasi guna memverifikasi limitasi bandwidth penyimpanan fisik [cite: 49, 53]:
2. Sistem kontainer pada pengujian Skenario Legacy harus menunjukkan kecepatan tulis yang konstan dan terbatas pada kisaran ≈ 10 MB/s [cite: 54].
3. **Pengujian Kinerja Komputasi Model AI:** Jalankan skrip inferensi lokal di dalam kontainer untuk merekam konsumsi RAM riil dan latensi eksekusi per gambar menggunakan utility internal [cite: 39, 55]. Jika model mengonsumsi memori melebihi batas 1 GB pada Skenario B, kernel Linux akan langsung mengirimkan sinyal interupsi Out-of-Memory (OOM) killer untuk menghentikan proses, yang menandakan perlunya konfigurasi model yang lebih ringan [cite: 48, 50].

Panduan Teknis Kompilasi dan Penyusutan Model AI

Di bawah ini dipaparkan alur instruksi pemrograman lengkap untuk mengeksport arsitektur DenseNet-121 dari PyTorch (TorchXRayVision) dan mengompresinya menggunakan teknik kuantisasi statis INT8 berbasis ONNX Runtime [cite: 22, 39, 42].

Bagian 1: Skrip Ekspor Model ke Graf ONNX

Tulis modul pengeksport berikut ke dalam file `export_model.py` [cite: 22, 56].

```
import torch
import torchxrayvision as xrv
```

```

def main():
    print("Memulai pemuatan model DenseNet-121 dari TorchXRayVision...")
    # Memuat model pra-latih yang mencakup cakupan patologi dada luas
    model = xrv.models.DenseNet(weights="densenet121-res224-all")
    model.eval()

    # Membuat dummy input tensor dengan resolusi 224x224 grayscale (1 saluran
    warna)
    dummy_input = torch.randn(1, 1, 224, 224)

    # Ekspor graf komputasi ke format standar ONNX dengan opset yang mendukung
    QDQ
    onnx_file_path = "densenet121_xrv.onnx"
    torch.onnx.export(
        model,
        dummy_input,
        onnx_file_path,
        export_params=True,
        opset_version=11, # Opset minimum untuk operasi kuantisasi modern
        do_constant_folding=True,
        input_names=["input_image"],
        output_names=["predictions"]
    )
    print(f"Ekspor model berhasil. File disimpan di: {onnx_file_path}")

if __name__ == "__main__":
    main()

```

Bagian 2: Skrip Kuantisasi Statis INT8 dengan Kalibrator Sampel

Penerapan kuantisasi statis mewajibkan adanya generator kalibrasi untuk menakar distribusi statistik nilai aktivasi secara akurat guna mempertahankan presisi diagnostik [cite: 39, 40]. Simpan instruksi berikut ke dalam file `quantize_model.py` [cite: 39, 42].

```

import numpy as np
from onnxruntime.quantization import quantize_static, CalibrationDataReader,
QuantFormat, QuantType

# Membuat kelas penyedia data kalibrasi representatif
class XRayCalibrationDataReader(CalibrationDataReader):
    def __init__(self, sample_size=100):
        # Menyiapkan citra radiografi acak yang dinormalisasi untuk emulasi
        data kalibrasi
        self.data_store = [np.random.randn(1, 1, 224, 224).astype(np.float32)

```

```

for _ in range(sample_size)]
    self.data_iterator = iter([{"input_image": sample} for sample in
self.data_store])

def get_next(self):
    # Mengembalikan dictionary input graf berikutnya
    return next(self.data_iterator, None)

def main():
    print("Memulai proses kalibrasi dan kuantisasi statis model...")
    calibration_reader = XRayCalibrationDataReader(sample_size=120)

    # Menjalankan mesin kuantisasi statis pasca-pelatihan (PTQ)
    quantize_static(
        model_input="densenet121_xrv.onnx",
        model_output="densenet121_xrv_int8.onnx",
        calibration_data_reader=calibration_reader,
        quant_format=QuantFormat.QDQ,          # Struktur QDQ teroptimasi murni
        untuk CPU
        activation_type=QuantType.QUInt8,      # Aktivasi tidak bertanda untuk
        kompatibilitas x86
        weight_type=QuantType.QInt8,           # Bobot bertanda 8-bit
        reduce_range=True                      # Mencegah saturasi matematika
        pada CPU faskes lama
    )
    print("Kuantisasi selesai. File terkompresi disimpan sebagai:
densenet121_xrv_int8.onnx")

if __name__ == "__main__":
    main()

```

Rekomendasi Integrasi Klinis dan Operasional Berkelanjutan

Optimalisasi arsitektur komputasi di atas server lokal harus diselaraskan dengan realitas implementasi klinis di lapangan agar mampu menghasilkan dampak kesehatan masyarakat yang berkesinambungan [cite: 4, 5].

Desain Aplikasi Modular Berbasis Offline-First

Mengingat infrastruktur jaringan intermiten di wilayah 3T, aplikasi diagnostik harus didesain dengan konsep *offline-first* [cite: 1, 4]. Server lokal Puskesmas menjalankan kontainer Docker model terkuantisasi secara mandiri dalam jaringan area lokal (LAN) klinik [cite: 8, 48].

Hasil tangkapan citra radiografi dari mesin X-ray portable langsung diunggah ke server lokal ini, diproses oleh model AI dalam waktu kurang dari satu detik, dan hasilnya segera ditampilkan di dasbor klinis [cite: 9, 21, 26]. Sinkronisasi data ke server rujukan pusat SatuSehat dilakukan

secara asinkron menggunakan sistem antrean ketika koneksi internet stabil atau terhubung secara periodik melalui layanan satelit [cite: 1, 4, 13].

Pendekatan Berpusat pada Manusia dan Pelatihan Berkelanjutan

Penerapan sistem medis bertenaga kecerdasan artifisial di daerah terpencil tidak ditujukan untuk menggantikan keputusan klinis dari tenaga medis lokal, melainkan untuk melipatgandakan kapabilitas penapisan awal mereka (*augmenting human judgment*) [cite: 4, 5, 25]. Masalah kurangnya keahlian radiologi dapat diredam dengan memberikan pelatihan berkala menggunakan metode hibrida (*blended learning*) kepada para perawat, bidan, dan dokter umum di Puskesmas [cite: 1, 4, 10].

Pelatihan difokuskan pada pengoperasian perangkat lunak secara ramah pengguna (*user-friendly*), pemahaman etika pengolahan data medis pasien, serta interpretasi visual keluaran kecerdasan artifisial yang disajikan dalam bentuk visualisasi peta perhatian panas (Grad-CAM) untuk meningkatkan transparansi klinis [cite: 21, 27, 57].

Penerapan Ambang Batas Triase Operasional Terstandarisasi

Untuk memaksimalkan keselamatan pasien dan menghemat pengeluaran rujukan yang tidak perlu, klasifikasi probabilitas numerik dari model AI harus dikonversikan ke dalam kategori rujukan taktis di Puskesmas [cite: 21, 58]. Berdasarkan analisis sebaran distribusi dari model TorchXRayVision, direkomendasikan penyetelan ambang batas keputusan operasional sebagai berikut [cite: 21]:

- **Probabilitas < 19% ("Insignifikan / Normal"):** Hasil tidak menunjukkan adanya indikasi anomali klinis mayor. Pasien dapat dipulangkan dengan aman atau diberikan edukasi preventif ringan, sehingga mencegah kepanikan dan menghemat anggaran operasional faskes [cite: 21, 58].
- **Probabilitas 20% - 30% ("Moderat / Pemantauan"):** Terdapat temuan anomali ringan. Nakes disarankan menjadwalkan pemeriksaan tindak lanjut dalam beberapa minggu ke depan atau melakukan intervensi klinis simptomatik [cite: 21].
- **Probabilitas > 31% ("Signifikan / Rujukan Segera"):** Indikasi kuat adanya infeksi paru aktif (seperti TB paru atau pneumonia berat) [cite: 21, 24, 25]. Pasien segera diprioritaskan untuk menjalani tes konfirmasi bakteriologis molekuler menggunakan mesin GeneXpert lokal atau langsung dirujuk ke rumah sakit kabupaten dengan dukungan rujukan berbasis data terkompresi [cite: 25, 28, 59].

Langkah-langkah taktis terpadu ini memadukan keunggulan teknologi kompresi pembelajaran mendalam dengan kearifan operasional lokal, menciptakan solusi penapisan yang andal, berkelanjutan, dan berkeadilan sosial bagi seluruh masyarakat Indonesia di daerah 3T [cite: 1, 4, 5].

1. Kesenjangan Digital di Pelayanan Kesehatan: Tantangan Manajemen Sistem Informasi di Daerah 3T - ResearchGate, https://www.researchgate.net/publication/392521620_Kesenjangan_Digital_di_Pelayanan_Kesehatan_Tantangan_Manajemen_Sistem_Informasi_di_Daerah_3T
2. Barriers in Tuberculosis Treatment in Rural Areas (Tengger, Osing and Pandalungan) in Indonesia Based on Public Health Center Professional Workers Perspectives: a Qualitative Research - Universitas Airlangga, <https://scholar.unair.ac.id/en/publications/barriers-in-tuberculosis-treatment-in-rural-areas-tengger-osing-a/>
3. Analysis of Artificial Intelligence Implementation in the Indonesian Healthcare Sector: A Literature Review - Journal IRPI, <https://www.journal.irpi.or.id/index.php/malcom/article/download/2229/1079>
4. Deploying medical AI in low-resource settings: a scoping review of challenges and strategies - Frontiers, <https://www.frontiersin.org/journals/digital-health/articles/10.3389/fdgth.2026.1743634/full>
5. (PDF) Deploying medical AI in low-resource settings: a scoping review of challenges and strategies - ResearchGate, https://www.researchgate.net/publication/403395836_Deploying_medical_AI_in_low-resource_settings_a_scoping_review_of_challenges_and_strategies
6. Frequently Asked Questions (FAQ) - - Healthical, <https://www.healthical.id/faq-2/>
7. APLIKASI SISTEM INFORMASI MANAJEMEN PUSKESMAS (SIMPUS) - aplikasisimkes, <https://aplikasisimkes.wordpress.com/2011/01/18/aplikasi-sistem-informasi-manajemen-puskesmas-simpus/>
8. Spesifikasi Kebutuhan Perangkat Lunak Sistem Informasi Penjangkaran Kesehatan Siswa Sekolah Dasar, <https://jurnal.umb.ac.id/index.php/JSAI/article/download/5268/3202>
9. Semarang City uses AI-powered mobile X-ray for TB screening - GovInsider, <https://govinsider.asia/intl-en/article/semarang-city-uses-ai-powered-mobile-x-ray-for-tb-screening>
10. Model of Integrated Surveillance System of Tuberculosis Based on the Internet of Things (IoT) for Accelerati - Griffith Research Online, <https://research-repository.griffith.edu.au/bitstreams/a045b2f1-f5e6-4be8-856a-3ce77ce2cee8/download>
11. GPU vs CPU for Computer Vision: AI Inference Optimization Guide - XenonStack, <https://www.xenonstack.com/blog/gpu-cpu-computer-vision-ai-inference>
12. Exploring the feasibility of real-time on-device ECG biometric classification using quantized neural networks - PMC, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12909525/>
13. Menkes Budi Mau Pakai Jaringan Satelit Starlink Buat Puskesmas di Wilayah 3T, <https://gizmologi.id/news/elon-musk-menkes-budi-jaringan-satelit-starlink/>

14. Project-MONAI/VLM-Radiology-Agent-Framework - GitHub, <https://github.com/Project-MONAI/VLM-Radiology-Agent-Framework>
15. GPU vs CPU for Healthcare ML Inference - Nirmitee.io, <https://nirmitee.io/blog/gpu-vs-cpu-healthcare-ml-inference-when-you-need-gpu/>
16. A Comprehensive Review of Knowledge Distillation for Lightweight Medical Image Segmentation | Journal of Advanced Health Informatics Research, <https://ejournal.ptti.web.id/index.php/jahir/article/view/294>
17. (PDF) TorchXRyVision: A library of chest X-ray datasets and models - ResearchGate, https://www.researchgate.net/publication/355841287_TorchXRyVision_A_library_of_chest_X-ray_datasets_and_models
18. Performance Comparison of Embedded AI Solutions for Classification and Detection in Lung Disease Diagnosis - MDPI, <https://www.mdpi.com/2076-3417/15/17/9345>
19. Project-MONAI/multi-modal - GitHub, <https://github.com/Project-MONAI/multi-modal>
20. MONAI Integrates Advanced Agentic Architectures to Establish Multimodal Medical AI Ecosystem | NVIDIA Technical Blog, <https://developer.nvidia.com/blog/monai-integrates-advanced-agentic-architectures-to-establish-multimodal-medical-ai-ecosystem/>
21. MCADS: Simultaneous Detection and Analysis of 18 Chest Radiographic Abnormalities Using Multi-Label Deep Learning - PMC, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12939440/>
22. Models — TorchXRyVision 1.0.1 documentation, <https://mlmed.org/torchxrayvision/models.html>
23. LTS-Net: A Lightweight and Efficient Deep Learning Framework for Automated Tuberculosis and Pneumonia Detection in Chest X-ray Images - Scilight Press, <https://www.sciltp.com/journals/aim/articles/2606004135>
24. A Lightweight and Efficient Deep Learning Framework for Automated Tuberculosis and Pneumonia Detection in Chest X-ray, <https://media.sciltp.com/articles/2606004135/2606004135.pdf>
25. Pulmonary tuberculosis prediction using CAD4TB artificial intelligence (computer-aided detection for tuberculosis) based on thoracic x-ray photos among Indonesian subjects in hospital | medRxiv, <https://www.medrxiv.org/content/10.1101/2025.04.20.25326134v1>
26. Partnering for Impact: Advancing TB Control in Indonesia - Delft Imaging, <https://delftimaging.com/resources/articles/partnering-for-impact-advancing-tb-control-in-indonesia/>
27. UGM Develops Indonesia's First AI-Powered Tuberculosis Screening App, <https://ugm.ac.id/en/news/ugm-develops-indonesias-first-ai-powered-tuberculosis-screening-app/>
28. Publication: Artificial intelligence in overcoming rifampicin resistant-screening challenges in Indonesia: a qualitative study on the user experience of CUHAS-ROBUST - Mahidol

IR, <https://repository.li.mahidol.ac.th/entities/publication/054f6910-d69a-4fde-9336-74801595f080>

29. Comparison of Deep Learning Approaches Using Chest Radiographs for Predicting Clinical Deterioration: Retrospective Observational Study - PMC, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12223691/>
30. HDKD: Hybrid Data-Efficient Knowledge Distillation Network for Medical Image Classification - arXiv, <https://arxiv.org/html/2407.07516v2>
31. Lightweight Model Adaptation for Mitigating Bias in Deep Learning Models for Chest X-Ray Analysis - CS231n - Stanford University, https://cs231n.stanford.edu/2025/papers/text_file_840073915-CS231N_final_report.pdf
32. Quantization of Deep Neural Networks for Medical Image Analysis: A Systematic Review and Meta-Analysis - MDPI, <https://www.mdpi.com/2227-7080/14/1/76>
33. Explainable Knowledge Distillation for Efficient Medical Image Classification - arXiv, <https://arxiv.org/html/2508.15251v1>
34. [2508.15251] Explainable Knowledge Distillation for Efficient Medical Image Classification - arXiv, <https://arxiv.org/abs/2508.15251>
35. Explainable Knowledge Distillation for Efficient Medical Image Classification, https://www.researchgate.net/publication/394831528_Explainable_Knowledge_Distillation_for_Efficient_Medical_Image_Classification
36. (PDF) Efficient Image Classification through Collaborative Knowledge Distillation: A Novel AlexNet Modification Approach - ResearchGate, https://www.researchgate.net/publication/382260001_Efficient_Image_Classification_through_Collaborative_Knowledge_Distillation_A_Novel_AlexNet_Modification_Approach
37. Detection of COVID-19, pneumonia, and tuberculosis from radiographs using AI-driven knowledge distillation - PMC, <https://pmc.ncbi.nlm.nih.gov/articles/PMC10912466/>
38. Intel Accelerates PadChest and fMRI Models on Azure ML, <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-accelerates-padchest-fmri-models-on-azure-ml.html>
39. Onnx Model Quantization | by Nashrakhan - Medium, <https://medium.com/@nashrakhan1008/model-quantization-8f10c537e0eb>
40. Quantize ONNX Models - ONNXRuntime - GitHub Pages, <https://iot-robotics.github.io/ONNXRuntime/docs/performance/quantization.html>
41. Quantize ONNX models | onnxruntime, <https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html>
42. onnxruntime/onnxruntime/python/tools/quantization/README.md at main - GitHub, <https://github.com/microsoft/onnxruntime/blob/main/onnxruntime/python/tools/quantization/README.md>

43. From Detection to Mitigation: Addressing Bias in Deep Learning Models for Chest X-Ray Diagnosis, <https://psb.stanford.edu/psb-online/proceedings/psb26/mottez.pdf>
44. Performance tuning - OpenVINO™ documentation, https://docs.openvino.ai/2024/openvino-workflow/model-server/ovms_docs_performance_tuning.html
45. Export YOLOv8 to OpenVINO for Fast Inference - Ultralytics, <https://www.ultralytics.com/blog/export-and-optimize-a-yolov8-model-for-inference-on-openvino>
46. onnxruntime/onnxruntime/python/tools/quantization/quantize.py at main · microsoft/onnxruntime - GitHub, <https://github.com/microsoft/onnxruntime/blob/main/onnxruntime/python/tools/quantization/quantize.py>
47. Edge Deep Learning in Computer Vision and Medical Diagnostics: A Comprehensive Survey - arXiv, <https://arxiv.org/html/2605.06714v1>
48. How to Implement Docker Container Resource Limits - OneUptime, <https://oneuptime.com/blog/post/2026-01-30-docker-container-resource-limits/view>
49. How to Set Up Docker Container Resource Constraints - OneUptime, <https://oneuptime.com/blog/post/2026-01-25-docker-container-resource-constraints/view>
50. How to Configure Docker Resource Limits - OneUptime, <https://oneuptime.com/blog/post/2026-02-02-docker-resource-limits/view>
51. A Comprehensive Guide to Monitoring Disk I/O on Linux - Last9, <https://last9.io/blog/monitoring-disk-io-on-linux/>
52. Rekomendasi Perangkat Terbaik untuk Pemakaian Sistem Klinik Assist.id, <https://blog.assist.id/rekomendasi-perangkat-pemakaian-sistem-klinik-assistid/>
53. Docker IOPS and Read/Write limitations not being applied - Server Fault, <https://serverfault.com/questions/839287/docker-iops-and-read-write-limitations-not-being-applied>
54. How to limit IO speed in docker and share file with system in the same time?, <https://stackoverflow.com/questions/36145817/how-to-limit-io-speed-in-docker-and-share-file-with-system-in-the-same-time>
55. Monitoring Container Resource Usage with Docker Stats - Dash0, <https://www.dash0.com/guides/docker-stats>
56. TorchXRyVision: A library of chest X-ray datasets and models - arXiv, <https://arxiv.org/pdf/2111.00595>
57. Explainable Knowledge Distillation for On-Device Chest X-Ray Classification, <https://www.computer.org/csdl/journal/tb/2024/04/10114588/1MPblzQvPZ6>
58. Closing the gap for real-world AI deployment in healthcare - MBZUAI, <https://mbzuai.ac.ae/news/closing-the-gap-for-real-world-ai-deployment-in->

[healthcare/](#)

59. Implementation costs and cost-effectiveness of ultraportable chest X-ray with artificial intelligence in active case finding for tuberculosis in Nigeria - PMC, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12157241/>